

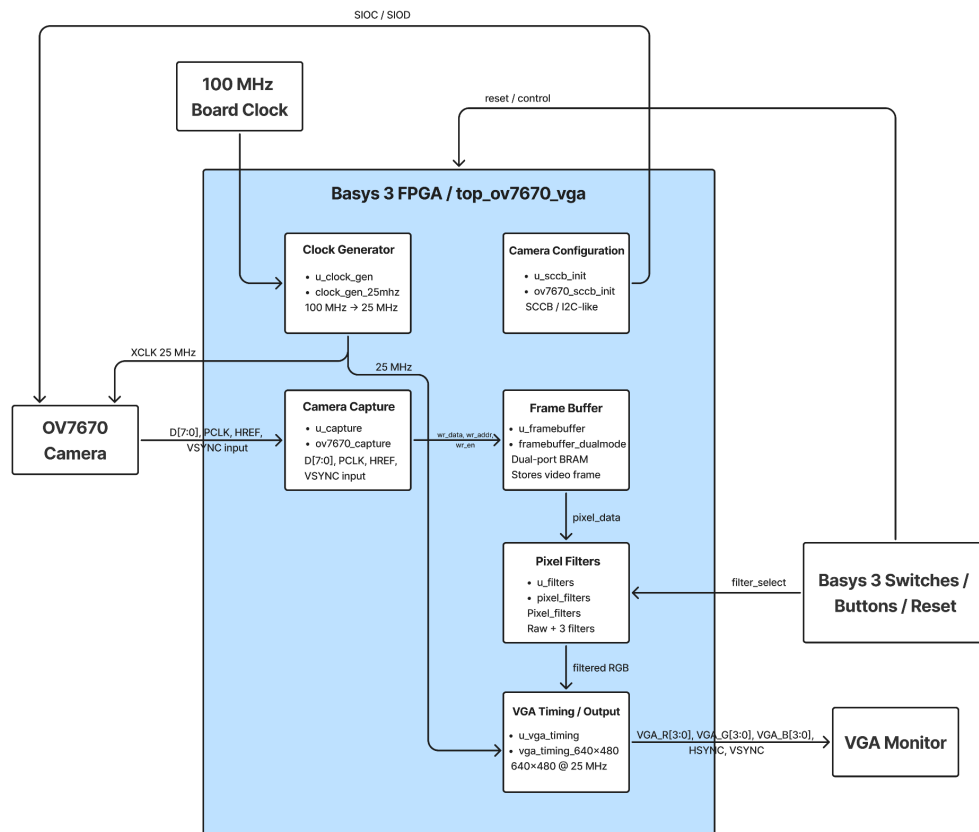
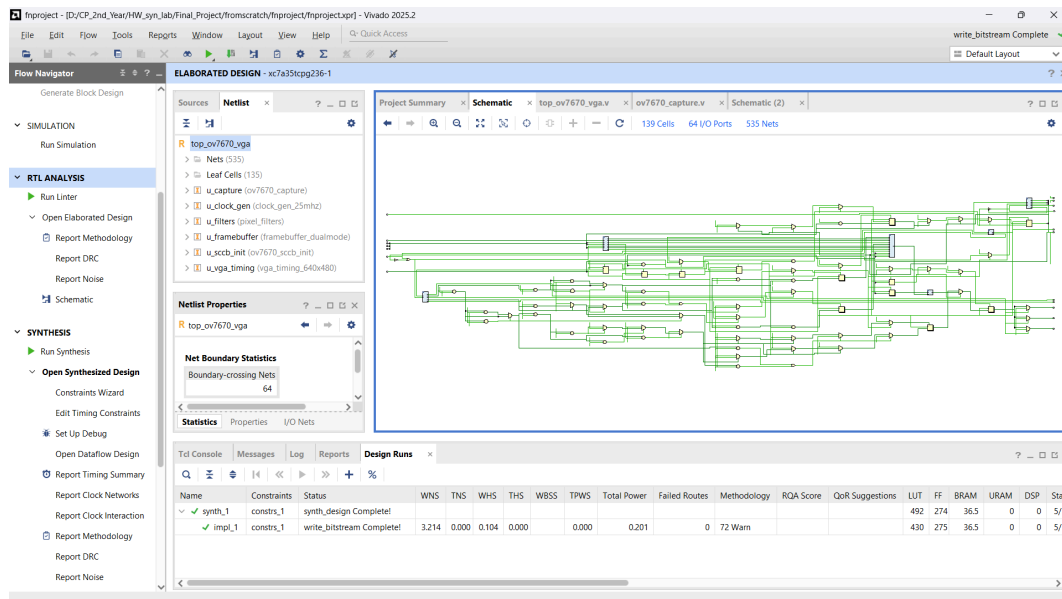
Group Member

Group name : กรีนเอเลี่ยน จรัญ

Name	Student Number
Jirapat Chaiya	6732007621
Nirawit Jongprasartsuk	6732022021
Passakorn Yangmak	6732029421
Metavin Karnjanarat	6730434521

AI Usage Disclosure : GPT5.5 was used to assist with report writing, design explanation, grammar refinement, and debugging discussion. The FPGA design implementation, hardware integration, testing, and verification were completed by the project group members.

Overall Design Block Diagram



Original picture :

<https://drive.google.com/drive/folders/1KvyivmU9GqYydHLOGHOjWJTcBEQTPiC0?usp=sharing>

Design Decision.

1. Module Architecture

- Design Decision: Independent Modular Architecture

- Why did you structure your design this way?

Answer: The design is separated into independent modules: camera configuration, camera capture, framebuffer memory, VGA timing, image filtering, and top-level control. This structure was chosen because each part of the video pipeline has a different responsibility and often operates in a different clock domain. For example, `ov7670_capture.v` works with the camera PCLK, while `vga_timing_640x480.v` and the framebuffer read side work with the 25 MHz VGA clock.

- What are the advantages of using this approach? (e.g., modularity, efficiency, ease of debugging)

Answer: The modular architecture makes the design easier to implement, test, and debug. Each module can be verified separately with a testbench before integration. The VGA timing, pixel filters, and OV7670 capture modules were tested independently in simulation. This also makes hardware debugging easier because problems can be separated into camera configuration, camera capture, memory storage, or VGA output.

2. Communication Protocols

- Design Decision: SCCB (Serial Camera Control Bus) Protocol with a Finite State Machine (FSM) and Lookup Table (LUT)

- Why did you choose this specific protocol (e.g., SPI, I2C, UART, AXI) for communication?

Answer: The OV7670 camera requires SCCB communication to configure its internal registers. SCCB is very similar to I2C and is used to set camera output format, clocking, windowing, exposure, gain, and color settings. Therefore, an SCCB controller was implemented in `ov7670_sccb_init.v`.

- How does this protocol suit your design requirements?

Answer: SCCB is only used during camera initialization, so it does not need high bandwidth. A 100 kHz SCCB clock is reliable and simple to generate from the 100 MHz Basys3 system clock. After initialization is complete, the camera sends image data through its parallel pixel bus, so SCCB does not limit the real-time video performance.

3. Clock and Data Transfer Rate

- Design Decision: 25 MHz Clock Generation via MMCM (Mixed-Mode Clock Manager)

- Why did you select this specific clock frequency?

Answer: Standard 640x480 VGA timing requires about a 25 MHz pixel clock. The Basys3 board provides a 100 MHz input clock, so the design uses an MMCM and BUFG to generate a stable 25 MHz clock for VGA timing and for the OV7670 XCLK.

- How does the data rate impact system performance?

Answer: VGA output must produce one pixel every 25 MHz clock cycle. If the framebuffer read path is delayed or misaligned, the image can shift or show edge artifacts. Because block RAM has synchronous read latency, the design prefetches framebuffer addresses ahead of the visible pixel. This keeps the displayed pixel aligned with the VGA timing.

- Did you consider constraints such as FPGA timing, external device compatibility, or noise margins?

Answer: Yes. The camera PCLK is constrained as a separate asynchronous clock because it returns from the OV7670 and is not phase-aligned with the FPGA-generated clock. Vivado reports show that the routed design meets timing with WNS = 3.214 ns and no DRC violations.

4. Resource Utilization

- Design Decision: Shared Dual-Mode Framebuffer using 3-bit Block RAM Banks

- How does your design balance resource usage (LUTs, FFs, BRAMs) and performance?

Answer: The design uses very little LUT and flip-flop logic because most processing is simple address generation, state machines, and combinational filtering. The main resource cost is block RAM, which is expected because the framebuffer is the largest part of the system.

- Why did you optimize certain parts of the design?

Answer: Block RAM is the limiting resource on the Basys3. The dual-mode framebuffer was designed so that both base color mode and full-resolution grayscale mode use the same 921,600-bit memory size. This allows the project to meet the base requirement and also provide an extra-credit full-resolution mode without exceeding FPGA resources.

Implementation Detail

Module : top_ov7670_vga

Module code :

```
module top_ov7670_vga (
    input  wire      CLK100MHZ,
    input  wire      btnC,
    input  wire [15:0] sw,
    output wire [15:0] led,

    input  wire [7:0]  cam_data,
    input  wire      cam_pclk,
    input  wire      cam_href,
    input  wire      cam_vsync,
    output wire      cam_xclk,
    output wire      cam_pwn,
    output wire      cam_reset,
    output wire      cam_sioc,
    inout  wire      cam_siod,

    output wire [3:0]  vga_r,
    output wire [3:0]  vga_g,
    output wire [3:0]  vga_b,
    output wire      vga_hsync,
    output wire      vga_vsync
);

localparam FULL_FRAME_WIDTH  = 640;
localparam FULL_FRAME_HEIGHT = 480;
localparam FULL_ADDR_WIDTH   = 19;
localparam BASE_FRAME_WIDTH  = 320;
localparam BASE_FRAME_HEIGHT = 240;
localparam BASE_ADDR_WIDTH   = 17;
localparam [9:0]  H_VISIBLE   = 10'd640;
localparam [10:0] H_TOTAL     = 11'd800;
localparam [9:0]  V_VISIBLE   = 10'd480;
localparam [10:0] V_TOTAL     = 11'd525;
localparam [10:0] FRAME_READ_LATENCY = 11'd3;
localparam [10:0] DEBUG_READ_LATENCY = 11'd2;

wire clk25;
wire clk25_locked;
wire rst = btnC || !clk25_locked;
```

```

    // Use a clocking resource instead of a fabric flip-flop divider. A divided
    // fabric clock can have enough skew/uncertainty to show up as edge
artifacts
    // when the VGA read path and camera write path interact.
    clock_gen_25mhz u_clock_gen (
        .clk100(CLK100MHZ),
        .rst    (btnC),
        .clk25  (clk25),
        .locked(clk25_locked)
    );

    assign cam_xclk  = clk25;
    assign cam_pwdn  = 1'b0;
    assign cam_reset = 1'b1;

    // SCCB (I2C-like) camera configuration
    wire sccb_sda_low;
    wire sccb_done;
    wire sccb_busy;
    wire sccb_ack_error;

    assign cam_siod = sccb_sda_low ? 1'b0 : 1'bz;

    ov7670_sccb_init u_sccb_init (
        .clk      (CLK100MHZ),
        .rst      (rst),
        .sioc     (cam_sioc),
        .siod_oe_low (sccb_sda_low),
        .siod_in   (cam_siod),
        .busy     (sccb_busy),
        .done     (sccb_done),
        .ack_error (sccb_ack_error)
    );

    // Camera capture path:
    // - full mode stores VGA 3-bit grayscale for the extra-credit path
    // - base mode stores downsampled QVGA RGB444 for better color
    wire          full_fb_wr_en;
    wire [FULL_ADDR_WIDTH-1:0] full_fb_wr_addr;
    wire [2:0]      full_fb_wr_data;
    wire          base_fb_wr_en;
    wire [BASE_ADDR_WIDTH-1:0] base_fb_wr_addr;
    wire [11:0]     base_fb_wr_data;
    wire          capture_frame_done_toggle;

```

```

ov7670_capture #(
    .FULL_FRAME_WIDTH  (FULL_FRAME_WIDTH),
    .FULL_FRAME_HEIGHT (FULL_FRAME_HEIGHT),
    .FULL_ADDR_WIDTH    (FULL_ADDR_WIDTH),
    .BASE_FRAME_WIDTH   (BASE_FRAME_WIDTH),
    .BASE_FRAME_HEIGHT  (BASE_FRAME_HEIGHT),
    .BASE_ADDR_WIDTH    (BASE_ADDR_WIDTH)
) u_capture (
    .pclk      (cam_pclk),
    .rst       (rst),
    .cam_data  (cam_data),
    .href      (cam_href),
    .vsync     (cam_vsync),
    .full_wr_en (full_fb_wr_en),
    .full_wr_addr(full_fb_wr_addr),
    .full_wr_data(full_fb_wr_data),
    .base_wr_en (base_fb_wr_en),
    .base_wr_addr(base_fb_wr_addr),
    .base_wr_data(base_fb_wr_data),
    .frame_done_toggle(capture_frame_done_toggle)
);

// VGA timing
wire [9:0] h_count;
wire [9:0] v_count;
wire      active_video;

vga_timing_640x480 u_vga_timing (
    .clk      (clk25),
    .rst      (rst),
    .h_count  (h_count),
    .v_count  (v_count),
    .hsync    (vga_hsync),
    .vsync    (vga_vsync),
    .active_video(active_video),
    .frame_start ()
);

// Read side from the shared dual-mode frame buffer.
reg  [FULL_ADDR_WIDTH-1:0] full_fb_rd_addr;
reg  [BASE_ADDR_WIDTH-1:0] base_fb_rd_addr;
wire [11:0]                fb_rd_rgb444;

reg  [1:0]                mode_d;
reg                                debug_mode_d;

```

```

reg [11:0]      debug_pixel;
reg            capture_frame_toggle_meta;
reg            capture_frame_toggle_sync;
reg            capture_frame_toggle_seen;
reg            resolution_mode_d;
reg [1:0]      camera_ready_count;
reg            camera_frame_ready;

// The VGA output register sees framebuffer data three clocks after the
// address is issued, so read ahead to keep the last visible columns
aligned.
wire [10:0] frame_h_sum = {1'b0, h_count} + FRAME_READ_LATENCY[10:0];
wire        frame_h_wrap = (frame_h_sum >= H_TOTAL[10:0]);
wire [10:0] frame_h_mod = frame_h_wrap ? (frame_h_sum - H_TOTAL[10:0]) :
frame_h_sum;
wire [9:0]  frame_v_next = (v_count == (V_TOTAL - 1)) ? 10'd0 : (v_count +
10'd1);
wire [9:0]  frame_h = frame_h_mod[9:0];
wire [9:0]  frame_v = frame_h_wrap ? frame_v_next : v_count;
wire        frame_prefetch_active = (frame_h < H_VISIBLE[9:0]) && (frame_v
< V_VISIBLE[9:0]);

// The generated debug pattern has one less register stage than the BRAM
path.
wire [10:0] debug_h_sum = {1'b0, h_count} + DEBUG_READ_LATENCY[10:0];
wire        debug_h_wrap = (debug_h_sum >= H_TOTAL[10:0]);
wire [10:0] debug_h_mod = debug_h_wrap ? (debug_h_sum - H_TOTAL[10:0]) :
debug_h_sum;
wire [9:0]  debug_v_next = (v_count == (V_TOTAL - 1)) ? 10'd0 : (v_count +
10'd1);
wire [9:0]  debug_h = debug_h_mod[9:0];
wire [9:0]  debug_v = debug_h_wrap ? debug_v_next : v_count;
wire        debug_prefetch_active = (debug_h < H_VISIBLE[9:0]) && (debug_v
< V_VISIBLE[9:0]);

wire [FULL_ADDR_WIDTH-1:0] full_row_512 = {1'b0, frame_v[8:0], 9'b0};
wire [FULL_ADDR_WIDTH-1:0] full_row_128 = {3'b000, frame_v[8:0], 7'b0};
wire [FULL_ADDR_WIDTH-1:0] full_col      = {9'b0000000000, frame_h};
wire [FULL_ADDR_WIDTH-1:0] full_fb_addr = full_row_512 + full_row_128 +
full_col;

wire [8:0] src_x = frame_h[9:1];
wire [7:0] src_y = frame_v[8:1];
wire [BASE_ADDR_WIDTH-1:0] base_row_256 = {1'b0, src_y, 8'b0};
wire [BASE_ADDR_WIDTH-1:0] base_row_064 = {3'b000, src_y, 6'b0};

```

```

wire [BASE_ADDR_WIDTH-1:0] base_col      = {8'b00000000, src_x};
wire [BASE_ADDR_WIDTH-1:0] base_fb_addr = base_row_256 + base_row_064 +
base_col;

wire [3:0] debug_stripe_phase = debug_h % 10'd12;
wire [1:0] debug_stripe_sel =
    (debug_stripe_phase < 4'd3) ? 2'd0 :
    (debug_stripe_phase < 4'd6) ? 2'd1 :
    (debug_stripe_phase < 4'd9) ? 2'd2 : 2'd3;
wire [11:0] debug_stripe_color =
    (debug_stripe_sel == 2'd0) ? 12'hF00 :
    (debug_stripe_sel == 2'd1) ? 12'h0F0 :
    (debug_stripe_sel == 2'd2) ? 12'h00F : 12'hFF0;

always @(posedge clk25) begin
    if (rst) begin
        full_fb_rd_addr <= {FULL_ADDR_WIDTH{1'b0}};
        base_fb_rd_addr <= {BASE_ADDR_WIDTH{1'b0}};
        mode_d          <= 2'b00;
        debug_mode_d     <= 1'b0;
        debug_pixel      <= 12'h000;
        capture_frame_toggle_meta <= 1'b0;
        capture_frame_toggle_sync <= 1'b0;
        capture_frame_toggle_seen <= 1'b0;
        resolution_mode_d <= 1'b0;
        camera_ready_count <= 2'd0;
        camera_frame_ready <= 1'b0;
    end else begin
        capture_frame_toggle_meta <= capture_frame_done_toggle;
        capture_frame_toggle_sync <= capture_frame_toggle_meta;

        if (resolution_mode_d != sw[14]) begin
            resolution_mode_d <= sw[14];
            capture_frame_toggle_seen <= capture_frame_toggle_sync;
            camera_ready_count <= 2'd0;
            camera_frame_ready <= 1'b0;
        end else if (capture_frame_toggle_sync !=
capture_frame_toggle_seen) begin
            capture_frame_toggle_seen <= capture_frame_toggle_sync;
            if (!camera_frame_ready) begin
                if (camera_ready_count == 2'd1) begin
                    camera_frame_ready <= 1'b1;
                end else begin
                    camera_ready_count <= camera_ready_count + 2'd1;
                end
            end
        end
    end
end

```

```

        end
    end

    if (frame_prefetch_active) begin
        full_fb_rd_addr <= full_fb_addr;
        base_fb_rd_addr <= base_fb_addr;
    end else begin
        full_fb_rd_addr <= {FULL_ADDR_WIDTH{1'b0}};
        base_fb_rd_addr <= {BASE_ADDR_WIDTH{1'b0}};
    end

    if (debug_prefetch_active) begin
        debug_pixel <= debug_stripe_color;
    end else begin
        debug_pixel <= 12'h000;
    end

    mode_d          <= sw[1:0];
    debug_mode_d     <= sw[15];
end

end

framebuffer_dualmode #(
    .FULL_ADDR_WIDTH(FULL_ADDR_WIDTH),
    .BASE_ADDR_WIDTH(BASE_ADDR_WIDTH)
) u_framebuffer (
    .wr_clk          (cam_pclk),
    .write_mode      (sw[14]),
    .full_wr_en      (full_fb_wr_en),
    .full_wr_addr    (full_fb_wr_addr),
    .full_wr_data    (full_fb_wr_data),
    .base_wr_en      (base_fb_wr_en),
    .base_wr_addr    (base_fb_wr_addr),
    .base_wr_data    (base_fb_wr_data),
    .rd_clk          (clk25),
    .read_mode       (sw[14]),
    .full_rd_addr    (full_fb_rd_addr),
    .base_rd_addr    (base_fb_rd_addr),
    .rd_rgb444       (fb_rd_rgb444)
);

wire [11:0] filtered_pixel;
pixel_filters u_filters (
    .pixel_in (fb_rd_rgb444),
    .mode     (mode_d),

```

```

        .pixel_out(filtered_pixel)
    );

    reg [11:0] video_pixel;
    always @(posedge clk25) begin
        if (rst) begin
            video_pixel <= 12'h000;
        end else begin
            if (debug_mode_d) begin
                video_pixel <= debug_pixel;
            end else if (!camera_frame_ready) begin
                video_pixel <= 12'h000;
            end else begin
                video_pixel <= filtered_pixel;
            end
        end
    end

    wire output_active = active_video;
    assign vga_r = output_active ? video_pixel[11:8] : 4'h0;
    assign vga_g = output_active ? video_pixel[7:4] : 4'h0;
    assign vga_b = output_active ? video_pixel[3:0] : 4'h0;

    assign led[0] = sccb_done;
    assign led[1] = sccb_busy;
    assign led[2] = cam_vsync;
    assign led[3] = cam_href;
    assign led[4] = cam_pclk;
    assign led[5] = sccb_ack_error;
    assign led[6] = sw[14]; // resolution mode: 0=base RGB444, 1=full
    grayscale
    assign led[7] = sw[15]; // debug pattern enable
    assign led[8] = camera_frame_ready;
    assign led[15:9] = 7'd0;

endmodule

module clock_gen_25mhz (
    input wire clk100,
    input wire rst,
    output wire clk25,
    output wire locked
);
`ifdef __ICARUS__
    reg [1:0] div;

```

```

reg          locked_reg;

always @(posedge clk100) begin
    if (rst) begin
        div          <= 2'b00;
        locked_reg <= 1'b0;
    end else begin
        div          <= div + 2'b01;
        locked_reg <= 1'b1;
    end
end

assign clk25  = div[1];
assign locked = locked_reg;
`else
wire clkfb;
wire clkfb_buf;
wire clk25_unbuf;

MMCME2_BASE #(
    .BANDWIDTH("OPTIMIZED"),
    .CLKFBOUT_MULT_F(10.0),
    .CLKFBOUT_PHASE(0.0),
    .CLKIN1_PERIOD(10.0),
    .CLKOUT0_DIVIDE_F(40.0),
    .CLKOUT0_DUTY_CYCLE(0.5),
    .CLKOUT0_PHASE(0.0),
    .DIVCLK_DIVIDE(1),
    .REF_JITTER1(0.010),
    .STARTUP_WAIT("FALSE")
) u_mmc (
    .CLKFBOUT (clkfb),
    .CLKFBOUTB(),
    .CLKOUT0  (clk25_unbuf),
    .CLKOUT0B (),
    .CLKOUT1  (),
    .CLKOUT1B (),
    .CLKOUT2  (),
    .CLKOUT2B (),
    .CLKOUT3  (),
    .CLKOUT3B (),
    .CLKOUT4  (),
    .CLKOUT5  (),
    .CLKOUT6  (),
    .LOCKED   (locked),

```

```

        .CLKFBIN   (clkfb_buf),
        .CLKIN1    (clk100),
        .PWRDWN    (1'b0),
        .RST       (rst)
    );

    BUFG u_clkfb_buf (
        .I(clkfb),
        .O(clkfb_buf)
    );

    BUFG u_clk25_buf (
        .I(clk25_unbuf),
        .O(clk25)
    );
`endif
endmodule

```

Module Description : The top_ov7670_vga module is the main integration module of the project. It connects the OV7670 camera interface, SCCB camera configuration, camera pixel capture, dual-mode framebuffer, VGA timing generator, image filters, switches, LEDs, and clock generator. This module controls the complete camera-to-VGA pipeline. The OV7670 sends pixel data using cam_pclk, cam_href, cam_vsync, and cam_data[7:0]. The capture module stores the received image into block RAM. The VGA side reads the framebuffer using a 25 MHz pixel clock and outputs the image to the VGA monitor. The module also supports debug pattern output, filter selection, and switching between base-resolution color mode and full-resolution grayscale mode.

Module : ov7670_sccb_init

Module code :

```

module ov7670_sccb_init #(
    parameter integer CLK_HZ          = 100_000_000,
    parameter integer SCCB_HZ         = 100_000,
    parameter integer POWERUP_DELAYMS = 20
) (
    input  wire clk,
    input  wire rst,
    output reg sioc,

```

```

output wire siod_oe_low,
input wire siod_in,
output reg busy,
output reg done,
output reg ack_error
);
localparam integer LUT_SIZE = 71;
localparam integer HALF_PERIOD = CLK_HZ / (SCCB_HZ * 2);
localparam integer STARTUP_TICKS = (CLK_HZ / HALF_PERIOD / 1000) *
POWERUP_DELAYMS;
localparam integer INTERREG_TICKS = (CLK_HZ / HALF_PERIOD / 10000); // ~100
us
localparam integer RESETREG_TICKS = (CLK_HZ / HALF_PERIOD / 100); // ~10
ms

localparam [3:0]
    ST_POWERUP_WAIT = 4'd0,
    ST_LOAD = 4'd1,
    ST_START_A = 4'd2,
    ST_START_B = 4'd3,
    ST_BIT_LOW = 4'd4,
    ST_BIT_HIGH = 4'd5,
    ST_ACK_LOW = 4'd6,
    ST_ACK_HIGH = 4'd7,
    ST_STOP_A = 4'd8,
    ST_STOP_B = 4'd9,
    ST_STOP_C = 4'd10,
    ST_REG_WAIT = 4'd11,
    ST_DONE = 4'd12;

reg [15:0] init_lut [0:LUT_SIZE-1];

reg [3:0] state;
reg [15:0] clk_div;
reg [23:0] wait_ticks;
reg [7:0] lut_idx;
reg [7:0] reg_addr;
reg [7:0] reg_data;
reg [7:0] tx_byte;
reg [1:0] byte_idx;
reg [2:0] bit_idx;
reg sda_drive_low;

wire tick = (clk_div == HALF_PERIOD - 1);

```

```

assign siod_oe_low = sda_drive_low;

initial begin
    // Soft reset
    init_lut[0] = 16'h1280;

    // Clocking / format / scaling for VGA RGB565.
    init_lut[1] = 16'h1101;
    init_lut[2] = 16'h6B4A;
    init_lut[3] = 16'h0C00;
    init_lut[4] = 16'h3E00;
    init_lut[5] = 16'h1204;
    init_lut[6] = 16'h8C00;
    init_lut[7] = 16'h40D0;
    init_lut[8] = 16'h3A04;

    // Windowing for VGA path. The previous horizontal window ended with a
    // repeated right-edge column on some OV7670 modules. These
HSTART/HSTOP
    // values are the common VGA preset used by many OV7670 reference
designs.
    init_lut[9] = 16'h32B6;
    init_lut[10] = 16'h1713;
    init_lut[11] = 16'h1801;
    init_lut[12] = 16'h1902;
    init_lut[13] = 16'h1A7A;
    init_lut[14] = 16'h030A;

    // Scaling control
    init_lut[15] = 16'h703A;
    init_lut[16] = 16'h7135;
    init_lut[17] = 16'h7211;
    init_lut[18] = 16'h73F0;
    init_lut[19] = 16'hA202;

    // Image tuning defaults
    init_lut[20] = 16'h1500;
    init_lut[21] = 16'h7A20;
    init_lut[22] = 16'h7B10;
    init_lut[23] = 16'h7C1E;
    init_lut[24] = 16'h7D35;
    init_lut[25] = 16'h7E5A;
    init_lut[26] = 16'h7F69;
    init_lut[27] = 16'h8076;
    init_lut[28] = 16'h8180;

```

```
init_lut[29] = 16'h8288;
init_lut[30] = 16'h838F;
init_lut[31] = 16'h8496;
init_lut[32] = 16'h85A3;
init_lut[33] = 16'h86AF;
init_lut[34] = 16'h87C4;
init_lut[35] = 16'h88D7;
init_lut[36] = 16'h89E8;

// Enable automatic exposure/gain/white balance. Starting at max
gain/AEC
// overexposes many OV7670 modules, especially under classroom
lighting.
// The banding filter and late AWB gain registers below reduce green
cast.

init_lut[37] = 16'h13EF;
init_lut[38] = 16'h0000;
init_lut[39] = 16'h1000;
init_lut[40] = 16'h0D40;
init_lut[41] = 16'h1418;
init_lut[42] = 16'hA505;
init_lut[43] = 16'hAB07;
init_lut[44] = 16'h2495;
init_lut[45] = 16'h2533;
init_lut[46] = 16'h26E3;

// Matrix coefficients
init_lut[47] = 16'h4FB3;
init_lut[48] = 16'h50B3;
init_lut[49] = 16'h5100;
init_lut[50] = 16'h523D;
init_lut[51] = 16'h53A7;
init_lut[52] = 16'h54E4;
init_lut[53] = 16'h589E;

// Misc recommended controls
init_lut[54] = 16'h3DC0;
init_lut[55] = 16'h1E37;
init_lut[56] = 16'h330B;
init_lut[57] = 16'h3C78;
init_lut[58] = 16'h6900;
init_lut[59] = 16'h7400;
init_lut[60] = 16'hB084;
init_lut[61] = 16'hB10C;
init_lut[62] = 16'hB20E;
```

```

init_lut[63] = 16'hB382;
init_lut[64] = 16'hB80A;

// Late AWB/color controls. Applying these after matrix/gamma setup
helps
// modules that otherwise make neutral white objects look green.
init_lut[65] = 16'h4108;
init_lut[66] = 16'h6F9F;
init_lut[67] = 16'h5500;
init_lut[68] = 16'h5640;
init_lut[69] = 16'h13EF;

// End marker
init_lut[70] = 16'hFFFF;
end

always @(posedge clk) begin
    if (rst) begin
        state          <= ST_POWERUP_WAIT;
        clk_div         <= 16'd0;
        wait_ticks      <= STARTUP_TICKS[23:0];
        lut_idx         <= 8'd0;
        reg_addr        <= 8'd0;
        reg_data        <= 8'd0;
        tx_byte         <= 8'h42;
        byte_idx        <= 2'd0;
        bit_idx         <= 3'd7;
        sioc            <= 1'b1;
        sda_drive_low   <= 1'b0;
        busy            <= 1'b1;
        done            <= 1'b0;
        ack_error       <= 1'b0;
    end else begin
        if (tick) begin
            clk_div <= 16'd0;

            case (state)
                ST_POWERUP_WAIT: begin
                    sioc          <= 1'b1;
                    sda_drive_low <= 1'b0;
                    busy          <= 1'b1;
                    done          <= 1'b0;

                    if (wait_ticks == 24'd0) begin
                        state <= ST_LOAD;
                    end
                end
            endcase
        end
    end
end

```

```

        end else begin
            wait_ticks <= wait_ticks - 24'd1;
        end
    end

ST_LOAD: begin
    if (init_lut[lut_idx] == 16'hFFFF) begin
        state <= ST_DONE;
    end else begin
        reg_addr      <= init_lut[lut_idx][15:8];
        reg_data      <= init_lut[lut_idx][7:0];
        tx_byte       <= 8'h42; // OV7670 write address
        byte_idx      <= 2'd0;
        bit_idx       <= 3'd7;
        state         <= ST_START_A;
    end
end

ST_START_A: begin
    sioc          <= 1'b1;
    sda_drive_low <= 1'b0;
    state         <= ST_START_B;
end

ST_START_B: begin
    sioc          <= 1'b1;
    sda_drive_low <= 1'b1;
    state         <= ST_BIT_LOW;
end

ST_BIT_LOW: begin
    sioc <= 1'b0;
    if (tx_byte[bit_idx]) begin
        sda_drive_low <= 1'b0;
    end else begin
        sda_drive_low <= 1'b1;
    end
    state <= ST_BIT_HIGH;
end

ST_BIT_HIGH: begin
    sioc <= 1'b1;
    if (bit_idx == 3'd0) begin
        state <= ST_ACK_LOW;
    end else begin

```

```

        bit_idx <= bit_idx - 3'd1;
        state <= ST_BIT_LOW;
    end
end

ST_ACK_LOW: begin
    sioc <= 1'b0;
    sda_drive_low <= 1'b0;
    state <= ST_ACK_HIGH;
end

ST_ACK_HIGH: begin
    sioc <= 1'b1;
    if (siod_in) begin
        ack_error <= 1'b1;
    end
    if (byte_idx == 2'd2) begin
        state <= ST_STOP_A;
    end else begin
        byte_idx <= byte_idx + 2'd1;
        bit_idx <= 3'd7;

        case (byte_idx)
            2'd0: tx_byte <= reg_addr;
            2'd1: tx_byte <= reg_data;
            default: tx_byte <= 8'h00;
        endcase

        state <= ST_BIT_LOW;
    end
end

ST_STOP_A: begin
    sioc <= 1'b0;
    sda_drive_low <= 1'b1;
    state <= ST_STOP_B;
end

ST_STOP_B: begin
    sioc <= 1'b1;
    sda_drive_low <= 1'b1;
    state <= ST_STOP_C;
end

ST_STOP_C: begin

```

```

        sioc            <= 1'b1;
        sda_drive_low <= 1'b0;
        if ((reg_addr == 8'h12) && (reg_data == 8'h80)) begin
            wait_ticks <= RESETREG_TICKS[23:0];
        end else begin
            wait_ticks <= INTERREG_TICKS[23:0];
        end
        state <= ST_REG_WAIT;
    end

    ST_REG_WAIT: begin
        if (wait_ticks == 24'd0) begin
            lut_idx <= lut_idx + 8'd1;
            state <= ST_LOAD;
        end else begin
            wait_ticks <= wait_ticks - 24'd1;
        end
    end

    ST_DONE: begin
        sioc            <= 1'b1;
        sda_drive_low <= 1'b0;
        busy            <= 1'b0;
        done            <= 1'b1;
        state           <= ST_DONE;
    end

    default: state <= ST_POWERUP_WAIT;
endcase
end else begin
    clk_div <= clk_div + 16'd1;
end
end
end
endmodule

```

Module Description : This module was designed as a finite state machine because SCCB communication is sequential. Each register write requires a start condition, device address, register address, data byte, ACK check, and stop condition. A lookup table is used so the camera register settings can be changed easily without modifying the SCCB transfer logic. This makes the design easier to debug because the protocol logic and camera configuration values are separated inside

Module : ov7670_capture

Module code :

```
module ov7670_capture #(
    parameter FULL_FRAME_WIDTH  = 640,
    parameter FULL_FRAME_HEIGHT = 480,
    parameter FULL_ADDR_WIDTH   = 19,
    parameter BASE_FRAME_WIDTH  = 320,
    parameter BASE_FRAME_HEIGHT = 240,
    parameter BASE_ADDR_WIDTH   = 17
) (
    input  wire          pclk,
    input  wire          rst,
    input  wire [7:0]    cam_data,
    input  wire          href,
    input  wire          vsync,

    output reg           full_wr_en,
    output reg [FULL_ADDR_WIDTH-1:0] full_wr_addr,
    output reg [2:0]     full_wr_data,

    output reg           base_wr_en,
    output reg [BASE_ADDR_WIDTH-1:0] base_wr_addr,
    output reg [11:0]    base_wr_data,

    output reg           frame_done_toggle
);

// Previous sync values are kept so the capture logic can detect line and
// frame edges from the camera timing signals.
reg href_d;
reg vsync_d;

// The OV7670 sends one RGB565 pixel as two 8-bit transfers. byte_phase
// marks whether the next byte is the first or second byte of the pixel.
reg byte_phase;
reg [7:0] pixel_hi;

// Pixel position and row base addresses for the two framebuffer formats.
reg [9:0] x_count;
reg [9:0] y_count;
reg [FULL_ADDR_WIDTH-1:0] full_row_addr;
reg [BASE_ADDR_WIDTH-1:0] base_row_addr;
reg frame_has_pixel;
```

```

// Edge detection for camera line/frame timing.
wire href_fall = href_d && !href;
wire vsync_rise = !vsync_d && vsync;

// Rebuild RGB565 components from the saved first byte and current second
byte.
wire [4:0] r5 = {pixel_hi[7:3]};
wire [5:0] g6 = {pixel_hi[2:0], cam_data[7:5]};
wire [4:0] b5 = {cam_data[4:0]};

// Compact brightness estimate used for the full-resolution grayscale mode.
// The near-black/near-white clamps help preserve contrast after reducing
to 3 bits.
wire [6:0] brightness = {2'b00, r5} + {2'b00, g6[5:1]} + {2'b00, b5};
wire      near_black = (brightness < 7'd18);
wire      near_white = (brightness > 7'd75);

// Base color mode stores a reduced RGB444 pixel.
wire [3:0] base_r4 = r5[4:1];
wire [3:0] base_g4 = g6[5:2];
wire [3:0] base_b4 = b5[4:1];

// Weighted grayscale approximation: 5R + 9G + 2B.
wire [7:0] gray_acc = (base_r4 << 2) + base_r4 +
                      (base_g4 << 3) + base_g4 +
                      (base_b4 << 1); // 5R + 9G + 2B, max 240
wire [2:0] full_gray = near_black ? 3'd0 :
                      (near_white ? 3'd7 : gray_acc[7:5]);

// Full mode keeps every 640x480 pixel. Base mode stores every other pixel
// and every other line to form a 320x240 color image.
wire capture_full = (x_count < FULL_FRAME_WIDTH) &&
                    (y_count < FULL_FRAME_HEIGHT);
wire capture_base = capture_full && !x_count[0] && !y_count[0] &&
                    (x_count[9:1] < BASE_FRAME_WIDTH) &&
                    (y_count[9:1] < BASE_FRAME_HEIGHT);

always @(posedge pclk) begin
    if (rst) begin
        href_d      <= 1'b0;
        vsync_d     <= 1'b0;
        byte_phase  <= 1'b0;
        pixel_hi    <= 8'd0;
        x_count     <= 10'd0;
        y_count     <= 10'd0;
    end
end

```

```

full_row_addr <= {FULL_ADDR_WIDTH{1'b0}};
base_row_addr <= {BASE_ADDR_WIDTH{1'b0}};
frame_has_pixel <= 1'b0;
full_wr_en      <= 1'b0;
full_wr_addr    <= {FULL_ADDR_WIDTH{1'b0}};
full_wr_data    <= 3'd0;
base_wr_en      <= 1'b0;
base_wr_addr    <= {BASE_ADDR_WIDTH{1'b0}};
base_wr_data    <= 12'd0;
frame_done_toggle <= 1'b0;
end else begin
    href_d      <= href;
    vsync_d     <= vsync;
    full_wr_en  <= 1'b0;
    base_wr_en  <= 1'b0;

    // Notify the VGA clock domain after a frame containing real pixels
ends.

if (vsync_rise && frame_has_pixel) begin
    frame_done_toggle <= ~frame_done_toggle;
    frame_has_pixel   <= 1'b0;
end

if (vsync) begin
    // Frame sync from camera. Start counting pixels from top-left.
    byte_phase <= 1'b0;
    x_count    <= 10'd0;
    y_count    <= 10'd0;
    full_row_addr <= {FULL_ADDR_WIDTH{1'b0}};
    base_row_addr <= {BASE_ADDR_WIDTH{1'b0}};
end else begin
    if (href) begin
        if (!byte_phase) begin
            // First byte of RGB565.
            pixel_hi <= cam_data;
            byte_phase <= 1'b1;
        end else begin
            // Second byte completes the pixel, so the write data
            // and framebuffer address can be generated.
            byte_phase <= 1'b0;

            if (capture_full) begin
                full_wr_en <= 1'b1;
                full_wr_addr <= full_row_addr + x_count;
                full_wr_data <= full_gray;
            end
        end
    end
end

```

```

        frame_has_pixel <= 1'b1;
    end

    if (capture_base) begin
        base_wr_en    <= 1'b1;
        base_wr_addr <= base_row_addr + x_count[9:1];
        base_wr_data <= {base_r4, base_g4, base_b4};
    end

    x_count <= x_count + 10'd1;
end
end

// End of active camera line. Move to the next row and update
// the row base addresses without using multipliers.
if (href_fall) begin
    byte_phase <= 1'b0;
    x_count    <= 10'd0;

    if (y_count < FULL_FRAME_HEIGHT) begin
        y_count <= y_count + 10'd1;
    end

    if (y_count < (FULL_FRAME_HEIGHT - 1)) begin
        full_row_addr <= full_row_addr + FULL_FRAME_WIDTH;
    end

    if (!y_count[0] && (y_count < (FULL_FRAME_HEIGHT - 2)))
        base_row_addr <= base_row_addr + BASE_FRAME_WIDTH;
    end
end
end
end
end
endmodule

```

Module Description : This module is responsible for converting the raw OV7670 byte stream into framebuffer writes. It uses HREF to know when pixel data is valid and VSYNC to restart frame counting. The design writes every pixel in full-resolution grayscale mode, but only every second pixel and second line in base color mode to create a 320x240 framebuffer.

Module : framebuffer_dualport

Module code :

```
module framebuffer_bank_3bit #(
    parameter ADDR_WIDTH = 17,
    parameter DEPTH      = 76800
) (
    input  wire          wr_clk,
    input  wire          wr_en,
    input  wire [ADDR_WIDTH-1:0] wr_addr,
    input  wire [2:0]      wr_data,

    input  wire          rd_clk,
    input  wire [ADDR_WIDTH-1:0] rd_addr,
    output reg [2:0]      rd_data
);
    // One 3-bit dual-clock RAM bank. The camera side writes with wr_clk,
    // and the VGA side reads with rd_clk.
    (* ram_style = "block" *) reg [2:0] mem [0:DEPTH-1];

    always @(posedge wr_clk) begin
        if (wr_en) begin
            mem[wr_addr] <= wr_data;
        end
    end

    always @(posedge rd_clk) begin
        rd_data <= mem[rd_addr];
    end
endmodule

module framebuffer_dualmode #(
    parameter FULL_ADDR_WIDTH = 19,
    parameter BASE_ADDR_WIDTH = 17
) (
    input  wire          wr_clk,
    input  wire          write_mode, // 0=base RGB444, 1=full
    grayscale
    input  wire          full_wr_en,
    input  wire [FULL_ADDR_WIDTH-1:0] full_wr_addr,
    input  wire [2:0]      full_wr_data,
    input  wire          base_wr_en,
    input  wire [BASE_ADDR_WIDTH-1:0] base_wr_addr,
    input  wire [11:0]      base_wr_data,
```

```

    input wire rd_clk,
    input wire read_mode, // 0=base RGB444, 1=full
    grayscale
    input wire [FULL_ADDR_WIDTH-1:0] full_rd_addr,
    input wire [BASE_ADDR_WIDTH-1:0] base_rd_addr,
    output wire [11:0] rd_rgb444
);
    localparam BANK_DEPTH = 76800; // 320 * 240

    // In full-resolution mode, four consecutive grayscale pixels share the
    // same bank address and are selected by the lower two address bits.
    wire [BASE_ADDR_WIDTH-1:0] full_wr_bank_addr =
full_wr_addr[FULL_ADDR_WIDTH-1:2];
    wire [BASE_ADDR_WIDTH-1:0] full_rd_bank_addr =
full_rd_addr[FULL_ADDR_WIDTH-1:2];

    wire full_bank0_sel = (full_wr_addr[1:0] == 2'd0);
    wire full_bank1_sel = (full_wr_addr[1:0] == 2'd1);
    wire full_bank2_sel = (full_wr_addr[1:0] == 2'd2);
    wire full_bank3_sel = (full_wr_addr[1:0] == 2'd3);

    // Base mode writes all four banks at the same address to form RGB444.
    // Full mode writes only one selected bank for each grayscale pixel.
    wire bank0_wr_en = write_mode ? (full_wr_en && full_bank0_sel) :
base_wr_en;
    wire bank1_wr_en = write_mode ? (full_wr_en && full_bank1_sel) :
base_wr_en;
    wire bank2_wr_en = write_mode ? (full_wr_en && full_bank2_sel) :
base_wr_en;
    wire bank3_wr_en = write_mode ? (full_wr_en && full_bank3_sel) :
base_wr_en;

    // The same physical bank address lines are reused for both formats.
    wire [BASE_ADDR_WIDTH-1:0] bank_wr_addr = write_mode ? full_wr_bank_addr :
base_wr_addr;
    wire [BASE_ADDR_WIDTH-1:0] bank_rd_addr = read_mode ? full_rd_bank_addr :
base_rd_addr;

    // Data packing differs by mode:
    // - base mode stores one RGB444 pixel across four banks
    // - full mode stores one 3-bit grayscale value in the selected bank
    wire [2:0] bank0_wr_data = write_mode ? full_wr_data : base_wr_data[11:9];
    wire [2:0] bank1_wr_data = write_mode ? full_wr_data : base_wr_data[8:6];
    wire [2:0] bank2_wr_data = write_mode ? full_wr_data : base_wr_data[5:3];

```

```

wire [2:0] bank3_wr_data = write_mode ? full_wr_data : base_wr_data[2:0];

wire [2:0] rd0;
wire [2:0] rd1;
wire [2:0] rd2;
wire [2:0] rd3;

// Four small 3-bit banks provide the same total storage for both modes:
// 320x240x12 in base mode or 640x480x3 in full mode.
framebuffer_bank_3bit #(
    .ADDR_WIDTH(BASE_ADDR_WIDTH),
    .DEPTH      (BANK_DEPTH)
) u_bank0 (
    .wr_clk (wr_clk),
    .wr_en  (bank0_wr_en),
    .wr_addr(bank_wr_addr),
    .wr_data(bank0_wr_data),
    .rd_clk (rd_clk),
    .rd_addr(bank_rd_addr),
    .rd_data(rd0)
);

framebuffer_bank_3bit #(
    .ADDR_WIDTH(BASE_ADDR_WIDTH),
    .DEPTH      (BANK_DEPTH)
) u_bank1 (
    .wr_clk (wr_clk),
    .wr_en  (bank1_wr_en),
    .wr_addr(bank_wr_addr),
    .wr_data(bank1_wr_data),
    .rd_clk (rd_clk),
    .rd_addr(bank_rd_addr),
    .rd_data(rd1)
);

framebuffer_bank_3bit #(
    .ADDR_WIDTH(BASE_ADDR_WIDTH),
    .DEPTH      (BANK_DEPTH)
) u_bank2 (
    .wr_clk (wr_clk),
    .wr_en  (bank2_wr_en),
    .wr_addr(bank_wr_addr),
    .wr_data(bank2_wr_data),
    .rd_clk (rd_clk),
    .rd_addr(bank_rd_addr),

```

```

        .rd_data(rd2)
    );

    framebuffer_bank_3bit #(
        .ADDR_WIDTH(BASE_ADDR_WIDTH),
        .DEPTH      (BANK_DEPTH)
    ) u_bank3 (
        .wr_clk (wr_clk),
        .wr_en  (bank3_wr_en),
        .wr_addr(bank_wr_addr),
        .wr_data(bank3_wr_data),
        .rd_clk (rd_clk),
        .rd_addr(bank_rd_addr),
        .rd_data(rd3)
    );

    reg      read_mode_d;
    reg [1:0] full_rd_sel_d;

    // Delay the mode and pixel selector to match the synchronous BRAM read
    data.
    always @(posedge rd_clk) begin
        read_mode_d    <= read_mode;
        full_rd_sel_d <= full_rd_addr[1:0];
    end

    // Full mode reconstructs RGB444 by expanding the selected 3-bit grayscale
    value.
    wire [2:0] full_gray =
        (full_rd_sel_d == 2'd0) ? rd0 :
        (full_rd_sel_d == 2'd1) ? rd1 :
        (full_rd_sel_d == 2'd2) ? rd2 : rd3;

    wire [3:0] full_gray4 = {full_gray, full_gray[2]};
    wire [11:0] full_rgb444 = {full_gray4, full_gray4, full_gray4};

    // Base mode directly combines the four banks into one RGB444 pixel.
    wire [11:0] base_rgb444 = {rd0, rd1, rd2, rd3};

    assign rd_rgb444 = read_mode_d ? full_rgb444 : base_rgb444;

endmodule

```

Module Description : The framebuffer_bank_3bit module is a small dual-clock block RAM bank that stores 3-bit pixel data. It uses separate write and read clocks. The camera side writes data using cam_pclk, while the VGA side reads data using the 25 MHz VGA clock. This allows the camera capture system and VGA display system to operate in different clock domains.

The framebuffer_dualmode module is the main image memory for the project. It uses four framebuffer_bank_3bit banks to store one video frame. The same physical block RAM is reused for two different display modes. In base mode, the framebuffer stores a 320x240 RGB444 image. Each pixel uses 12 bits, so the red, green, and blue data are distributed across the four 3-bit memory banks. In full-resolution mode, the framebuffer stores a 640x480 grayscale image. Since each grayscale pixel only uses 3 bits, four consecutive grayscale pixels can be packed into the four memory banks at the same address.

This design saves block RAM because both modes use the same total memory size:

$$320 \times 240 \times 12 = 921,600 \text{ bits}$$

$$640 \times 480 \times 3 = 921,600 \text{ bits}$$

The module outputs all data as RGB444 so the rest of the VGA pipeline can use the same pixel format in both modes.

Module : vga_timing_640x480

Module code :

```
module vga_timing_640x480 (  
    input wire      clk,  
    input wire      rst,  
    output reg [9:0] h_count,  
    output reg [9:0] v_count,  
    output wire      hsync,  
    output wire      vsync,  
    output wire      active_video,  
    output wire      frame_start  
);
```

```

// Standard 640x480 VGA horizontal timing at a 25 MHz pixel clock.
localparam H_VISIBLE = 10'd640;
localparam H_FRONT   = 10'd16;
localparam H_SYNC    = 10'd96;
localparam H_BACK    = 10'd48;
localparam H_TOTAL   = H_VISIBLE + H_FRONT + H_SYNC + H_BACK; // 800

// Standard 640x480 VGA vertical timing.
localparam V_VISIBLE = 10'd480;
localparam V_FRONT   = 10'd10;
localparam V_SYNC    = 10'd2;
localparam V_BACK    = 10'd33;
localparam V_TOTAL   = V_VISIBLE + V_FRONT + V_SYNC + V_BACK; // 525

// Pixel counters scan across one line, then advance to the next line.
always @(posedge clk) begin
    if (rst) begin
        h_count <= 10'd0;
        v_count <= 10'd0;
    end else begin
        if (h_count == H_TOTAL - 1) begin
            h_count <= 10'd0;
            if (v_count == V_TOTAL - 1) begin
                v_count <= 10'd0;
            end else begin
                v_count <= v_count + 10'd1;
            end
        end else begin
            h_count <= h_count + 10'd1;
        end
    end
end

// VGA sync pulses are active low.
assign hsync = ~(h_count >= (H_VISIBLE + H_FRONT)) &&
              (h_count < (H_VISIBLE + H_FRONT + H_SYNC));

assign vsync = ~(v_count >= (V_VISIBLE + V_FRONT)) &&
              (v_count < (V_VISIBLE + V_FRONT + V_SYNC));

// active_video is high only inside the visible 640x480 region.
assign active_video = (h_count < H_VISIBLE) && (v_count < V_VISIBLE);
assign frame_start  = (h_count == 10'd0) && (v_count == 10'd0);

endmodule

```

Module Description : The vga_timing_640x480 module generates the VGA timing signals for a 640x480 display. It uses horizontal and vertical pixel counters to track the current screen position and generates active-low hsync and vsync signals based on standard VGA timing. The module also outputs active_video, which becomes high only during the visible 640x480 display area, and frame_start, which indicates the first pixel of a new frame.

Module : pixel_filter

Module code :

```
module pixel_filters (
    input wire [11:0] pixel_in,
    input wire [1:0] mode,
    output reg [11:0] pixel_out
);
    // Split RGB444 input into separate color channels.
    wire [3:0] r = pixel_in[11:8];
    wire [3:0] g = pixel_in[7:4];
    wire [3:0] b = pixel_in[3:0];

    // Weighted grayscale approximation. Green is weighted highest because it
    // contributes most to perceived brightness.
    wire [7:0] gray_acc = (r << 2) + r + (g << 3) + g + (b << 1); // 5R + 9G +
2B
    wire [3:0] gray      = gray_acc[7:4];

    // Switch-selected real-time filter.
    always @(*) begin
        case (mode)
            2'b00: pixel_out = pixel_in;                // Raw
            2'b01: pixel_out = {gray, gray, gray};       // Grayscale
            2'b10: pixel_out = {~r, ~g, ~b};             // Negative
            2'b11: pixel_out = {r, 4'h0, 4'h0};          // Red-only channel
            default: pixel_out = pixel_in;
        endcase
    end
endmodule
```

Module Description : The pixel_filters module applies a switch-selected real-time filter to one RGB444 pixel. It supports raw video, grayscale, negative, and red-only modes. The filter is combinational, so it does not store image data and does not add a separate clocked pipeline stage. This keeps the logic small and allows the displayed filter to change immediately when sw[1:0] changes.

Challenge Faced

1. OV7670 camera configuration

The camera required SCCB register setup before it could output a correct image. Wrong register values caused bad color, wrong brightness, or alignment issues.

2. Green tint and brightness issue

The image was sometimes too bright or green-tinted. This was improved by adjusting exposure, white balance, and color-related camera registers.

3. Right-side vertical stripe

A stripe appeared on the right edge of the screen. Debug patterns showed that VGA timing was likely correct, so the problem was probably related to camera windowing or capture alignment.

4. Framebuffer memory limit

The Basys3 did not have enough block RAM for full 640x480 color. The solution was to use 320x240 RGB color for normal mode and 640x480 3-bit grayscale for extra-credit mode.

5. Debugging hardware issues

FPGA debugging was difficult because internal signals were not directly visible. LEDs, switches, simulation, and VGA debug patterns were used to identify problems.